

JavaScript



The Language of the Web

Computer Science — 1st Year Presentation



Group Members

Boualem Bayoud | Anis Kara

Ahmed Amine Abd Elssalam | Abd El Djalil Adjou



University of Bouira | March 27, 2026

What is JavaScript? > Overview & Key Characteristics

JavaScript is a **high-level**, **interpreted** and **dynamic** programming language — the **only** language that runs natively inside every web browser.

Front-End

Makes web pages **interactive**, animated and responsive — without reloading from the server.

Back-End

Runs on servers via **Node.js** to handle databases, APIs and real-time applications.

Mobile & Desktop

Cross-platform apps via **React Native** (mobile) and **Electron** (desktop).

Key Facts

- ✓ Used by **98%** of all websites worldwide
- ✓ Runs in every modern browser — *no installation needed*
- ✓ Part of the essential **HTML / CSS / JS** trio
- ✓ **#1** most used language on GitHub for years
- ✓ Huge ecosystem: **React, Vue, Angular, Node.js**



script.js

History of JavaScript ▶ From 10 Days of Work to the World's Most Used Language

1995

Brendan Eich creates **Mocha** (later re-named **JavaScript**) at *Netscape* in just **10 days**.

1996

Microsoft releases **JScript** for Internet Explorer — the “browser wars” begin.

1997

Standardised as **ECMAScript** by ECMA International — one specification for all browsers.

2009

Node.js released — JavaScript now runs on *servers*, not just browsers.

Now

#1 language on GitHub. Powering the entire modern web.

Creator of JavaScript



Brendan Eich

-  Born: July 4, 1961
-  American programmer
-  Co-founder of Mozilla
-  Created JavaScript in May 1995
-  Built in only 10 days!

💡 Why was JavaScript created?

Netscape needed a **lightweight scripting language** so that web designers — not just programmers — could make pages **interactive** without reloading them from the server every time.



CHAPTER 2

Control & Logic

Conditions ▪ Loops ▪ Functions ▪ Frameworks



Abdesslam Ahmed Amine



Introduction to Artificial Intelligence

Chapter Overview > What we will cover in the next ~6 minutes



Conditions

Deciding what
code runs



Loops

Repeating
actions



Functions

Reusable
blocks of code



Frameworks

React &
Node.js



Why does this matter?

Control structures are the **skeleton** of every program. Without them, code would just run line-by-line with no decisions, no repetition, and no reuse.

Conditions — `if / else` ▶ Making decisions in code

- ▶ Code runs **only when** a condition is

`true`

- ▶ `else if` chains multiple conditions

- ▶ `else` is the final fallback

- ▶ Conditions use **comparison operators**:

`===` `!==` `>` `<` `>=` `<=`

⚠ Remember

Always use `===` (strict equality),
not `==` (loose equality).

```
1  const grade = 14;
2  if (grade >= 16) {
3    console.log("Excellent!");
4  } else if (grade >= 10) {
5    console.log("Passed");
6  } else {
7    console.log("Failed");
8  }
9  // Output: "Passed"
```

Conditions — Ternary & Switch › Shorter syntax for simple choices

Ternary Operator

```
1 // condition ? valueA : valueB
2 const age = 20;
3 const status = age >= 18
4   ? "Adult"
5   : "Minor";
6 console.log(status); // "Adult"
```

- ❗ Best for **simple, one-line** decisions.
- Avoid nesting ternaries — hard to read.

Switch Statement

```
1 const day = "Monday";
2 switch (day) {
3   case "Monday":
4     console.log("Week starts!");
5     break;
6   case "Friday":
7     console.log("Weekend soon!");
8     break;
9   default:
10    console.log("Regular day");
11 }
```

- ❗ `break` stops fall-through.

Loops — The for Loop

Repeat an action a known number of times

A `for` loop has 3 parts:

> **Initializer** — `let i = 0`

Runs once at the start

> **Condition** — `i < 5`

Checked before each iteration

> **Update** — `i++`

Runs after each iteration

✓ Output

1 2 3 4 5

```
1 // Count from 1 to 5
2 for (let i = 1; i <= 5; i++) {
3   console.log(i);
4 }
5 // Loop over an array
6 const students = [
7   "Ali", "Sara", "Youssef"
8 ];
9 for (let i = 0;
10      i < students.length; i++) {
11   console.log(students[i]);
12 }
```


Loops — while & for...of ➤ Two more ways to repeat code

while — Unknown iterations

```
1 let count = 0;
2 while (count < 3) {
3   console.log("Count: " + count);
4   count++;
5 }
6 // Count: 0
7 // Count: 1
8 // Count: 2
```

⚠ Always update the variable inside the loop or it runs forever!

for...of — Clean array iteration

```
1 const fruits = [
2   "apple", "mango", "fig"
3 ];
4 for (const fruit of fruits) {
5   console.log(fruit);
6 }
7 // apple
8 // mango
9 // fig
```

✓ Cleaner than `for` when you don't need the index. Preferred modern JS.

Functions — Reusable Blocks of Code > Write once, use anywhere

- > A function is a **named block** of code
- > It runs only when **called**
- > It can receive **parameters** (inputs)
- > It can **return** a result



Two declaration styles

Declaration: can be used before it is defined (hoisted)

Expression: stored in a variable

```
1 // Function Declaration
2 function greet(name) {
3   return "Hello, " + name + "!";
4 }
5 console.log(greet("Ahmed"));
6 // Hello, Ahmed!
7
8 // Function Expression
9 const square = function(n) {
10   return n * n;
11 };
12 console.log(square(5)); // 25
```

Functions — Arrow Functions (ES6) ➤ A shorter, modern syntax introduced in 2015

- Introduced in **ES6 (2015)**
- Shorter syntax with `=>`
- No `function` keyword needed
- One-liners can skip `return` & braces
- Very common in modern codebases

💡 Rule of thumb

Use arrow functions for short, simple tasks.

Use regular functions for complex logic.

```
1 // Regular function
2 function add(a, b) {
3   return a + b;
4 }
5 // Same - arrow style
6 const add = (a, b) => a + b;
7 console.log(add(3, 4)); // 7
8
9 // Multi-line arrow function
10 const describe = (name, age) => {
11   const msg =
12     `${name} is ${age} years old.`;
13   return msg;
14 };
15 console.log(describe("Sara", 20));
16 // Sara is 20 years old.
```

? What is a Framework?

A framework is a **collection of pre-written code** that gives you a structure to build applications faster, without starting from zero.



React

Runs in the **browser**
(Front-end / Client-side)

Built by Meta (Facebook)
Used for building user interfaces
Component-based architecture

VS



Node.js

Runs on the **server**
(Back-end / Server-side)

Built on Chrome's V8 engine
Used for APIs & web servers
Non-blocking, event-driven

React — Building User Interfaces > The most popular front-end library in the world

- > Created by **Meta** in 2013
- > Builds UIs from small reusable pieces called **components**
- > Uses **JSX** — HTML-like syntax inside JavaScript
- > Updates only the parts of the page that changed (**Virtual DOM**)

★ Used by

Facebook, Instagram, Airbnb,
Netflix, WhatsApp Web

```
1 // A simple React component
2 function WelcomeCard({ name }) {
3   return (
4     <div className="card">
5       <h2>Hello, {name}!</h2>
6       <p>Welcome to our app.</p>
7     </div>
8   );
9 }
10 // Using the component
11 function App() {
12   return <WelcomeCard name="Ahmed" />;
13 }
```

i JSX looks like HTML but it is JavaScript under the hood.

Node.js — JavaScript on the Server > Running JavaScript outside the browser

- Created by **Ryan Dahl** in 2009
- Lets JS run on a **server**, not just a browser
- Powers REST APIs, chat apps, real-time tools
- Uses **npm** — the world's largest package registry
- Non-blocking: handles many users **simultaneously**

★ Used by

LinkedIn, Netflix, Uber, PayPal, NASA

```
1 // A simple web server with Node.js
2 const http = require("http");
3 const server = http.createServer(
4   (request, response) => {
5     response.writeHead(200, {
6       "Content-Type": "text/plain"
7     });
8     response.end(
9       "Hello from Node.js!"
10    );
11  }
12 );
13 server.listen(3000, () => {
14   console.log(
15     "Server running on port 3000"
16   );
17 });
```

Chapter 2 — Summary > Key takeaways

Conditions (`if` / `else` , `switch`)

Let your program make decisions

Loops (`for` , `while` , `for...of`)

Repeat code efficiently

Functions (declaration, expression, arrow)

Write once, reuse everywhere

Frameworks (React, Node.js)

Build real applications faster

→ What comes next

Chapter 3 will cover:

- > Interacting with HTML (DOM)
- > Handling events
- > Common uses of JavaScript
- > Advantages & limitations